In [1]:
```
pip install numpy matplotlib scikit-learn
```

Requirement already satisfied: numpy in ./miniconda3/lib/python3.13/site-p ackages (2.4.2)
Requirement already satisfied: matplotlib in ./miniconda3/lib/python3.13/s ite-packages (3.10.8)
Requirement already satisfied: scikit-learn in ./miniconda3/lib/python3.1 3/site-packages (1.8.0)
Requirement already satisfied: contourpy>=1.0.1 in ./miniconda3/lib/python 3.13/site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in ./miniconda3/lib/python3.1 3/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in ./miniconda3/lib/pytho n3.13/site-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in ./miniconda3/lib/pytho n3.13/site-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in ./miniconda3/lib/python 3.13/site-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in ./miniconda3/lib/python3.13/si te-packages (from matplotlib) (12.0.0)
Requirement already satisfied: pyparsing>=3 in ./miniconda3/lib/python3.1 3/site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in ./miniconda3/lib/py thon3.13/site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: scipy>=1.10.0 in ./miniconda3/lib/python3.1 3/site-packages (from scikit-learn) (1.17.1)
Requirement already satisfied: joblib>=1.3.0 in ./miniconda3/lib/python3.1 3/site-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in ./miniconda3/lib/py thon3.13/site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: six>=1.5 in ./miniconda3/lib/python3.13/sit

In [2]:
e-packages (from python-dateutil>=2.7->matplotlib) (1.17.0) Note: you may need to restart the kernel to use updated packages.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Dataset
X = np.array([
 [2, 10], # A1
 [2, 5], # A2
 [8, 4], # A3
 [5, 8], # B1
 [7, 5], # B2
 [6, 4], # B3
 [1, 2], # C1
 [4, 9] # C2
])

labels = ['A1', 'A2', 'A3', 'B1', 'B2', 'B3', 'C1', 'C2']

# Initial centroids: A1, B1, C1
initial_centers = np.array([
 [2, 10], # A1
 [5, 8], # B1
 [1, 2] # C1
])
```

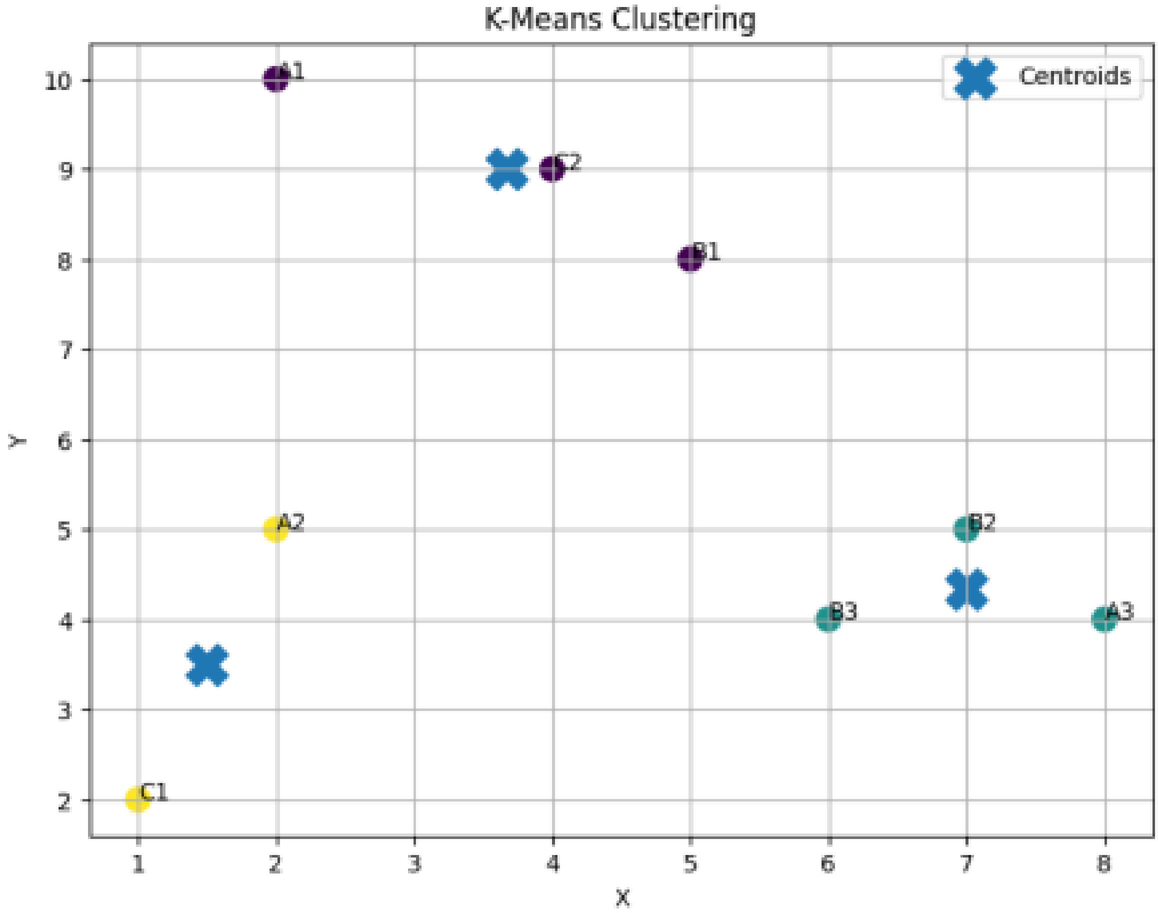


```
# K-Means with specified initial centroids
kmeans = KMeans(
 n_clusters=3,
 init=initial_centers,
 n_init=1,
 random_state=0
)


"""
# K-Means Clustering (K = 3)
kmeans = KMeans(
 n_clusters=3,
 random_state=42,
 n_init=10
)
```

```
"""

kmeans.fit(X)

cluster_labels = kmeans.labels_
centroids = kmeans.cluster_centers_

# Display results
print("Cluster Assignments:")
for i in range(len(X)):
                                        print(f"{labels[i]} {tuple(X[i])} --> Cluster {cluster_labels[i]}")

print("\nFinal Centroids:")
print(centroids)

# Plot
plt.figure(figsize=(8,6))
plt.scatter(X[:,0], X[:,1], c=cluster_labels, s=100)

for i, label in enumerate(labels):
 plt.annotate(label, (X[i,0], X[i,1]))

plt.scatter(
 centroids[:,0],
 centroids[:,1],
 marker='X',
 s=300,
 label='Centroids'
)

plt.title("K-Means Clustering")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.grid(True)
plt.show()
```



```
Cluster Assignments:
A1 (np.int64(2), np.int64(10)) --> Cluster 0
A2 (np.int64(2), np.int64(5)) --> Cluster 2
A3 (np.int64(8), np.int64(4)) --> Cluster 1
B1 (np.int64(5), np.int64(8)) --> Cluster 0
B2 (np.int64(7), np.int64(5)) --> Cluster 1
B3 (np.int64(6), np.int64(4)) --> Cluster 1
C1 (np.int64(1), np.int64(2)) --> Cluster 2
C2 (np.int64(4), np.int64(9)) --> Cluster 0

Final Centroids:
[[3.66666667 9.        ]
 [7.         4.33333333]
 [1.5        3.5       ]]
```


K-Means Clustering
Centroids
A1
C2
B1
A2
B2
B3
A3
C1
X
Y
1
2
3
4
5
6
7
8
10
9
8
7
6
5
4
3
2

In [ ]: